SIMATS ENGINEERING

Department of Information Technology

DSA0114 – Object-Oriented Programming with C++

ASSIGNMENT 2 – CO2

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs.

Student Name	Maddipati Teja	**Register Number**	192425323
Department / Section	AIML	**Academic Year / Semester**	3rd year
Faculty Name	Amirtha	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	06-11-2026	**Date of Evaluation**	
GitHub Repository	DSA0114 C++ M Teja - 192425323	**Compiler**	Wextra

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		
9	Implementation and Testing	4		

Task	Task Title	Max	Marks Awarded	Remarks
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALL REMARKS

M Teja		
Student Signature	**Faculty Signature**	**Program Director**

Q1 – Call by Value, by Reference and by Pointer

1. Three implementations of the hostel-fee concession function

```
#include <iostream>
using namespace std;

void byValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}

void byReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}

void byPointer(long long* fee, int percent) {
    if (fee != nullptr)
        *fee -= (*fee) * percent / 100;
}

int main() {
    long long fee1 = 45000, fee2 = 45000, fee3 = 45000;
    byValue(fee1, 10);
    byReference(fee2, 10);
    byPointer(&fee3, 10);
    cout << fee1 << " " << fee2 << " " << fee3 << '\\n';
}
```

Predicted and verified result: 45000 40500 40500. The value parameter is a separate local copy, so the original fee1 is unchanged. The reference parameter is an alias for fee2, and the pointer points to fee3; changing the referenced/dereferenced object changes the caller's variable.

2. Where parameters are stored and const reference

- Call by value: a local parameter copy is created. For a long long this is a small value copy.
- Call by reference: the parameter is an alias to the caller's object; no separate object is copied.
- Call by pointer: the pointer parameter receives an address. Dereferencing it accesses the caller's object. A null check avoids dereferencing a null pointer.
- Use const T& when a function needs to read a potentially large object without copying it and must not modify it.

3. Design decision for multiple concessions

For a fee update that is part of the student record, an in-place reference-based function is suitable because the function clearly updates the existing fee. The function should validate the percentage and apply each concession in a documented order. If the calculation should be pure and side-effect-free, returning the new fee is safer because the caller explicitly assigns the result.

```
long long applyConcession(long long fee, int percent) {
    return fee - fee * percent / 100;
}
// fee = applyConcession(fee, 10);
```

4. Predict the output of the supplied program

1 12 13

```
1 100
16 13 6
```

Explanation: change(a,b,&c) increments only its local copy a, but changes b through its reference and c through its pointer. Therefore a=1, b=12, c=13. bigger(a,c) returns a reference to the larger variable. Since c=13 is larger, the assignment bigger(a,c)=100 changes c to 100; a remains 1. show(1) uses y=5 and z=10, giving 16. show(1,2) uses z=10, giving 13—not 8. show(1,2,3) gives 6. Correct exact output is therefore:

```
1 12 13
1 100
16 13 6
```

The first displayed line is 1 12 13, because the value parameter a does not change the caller. The second line is 1 100, because bigger returns a reference to c. Default arguments used: show(1) uses 5 and 10; show(1,2) uses 10; show(1,2,3) uses no defaults.

5. Complete the code – call by reference

```
void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
```

Expected output:

```
after call by value : 45000
after call by reference: 40500
```

6. Complexity and alternative

Parameter design	Time	Extra parameter storage / copying
long long by value	O(1)	One long long copy
long long by reference	O(1)	Reference/alias; no object copy
Student by value	O(1) relative to fixed-size object; O(m) if it contains m elements	Copies the whole object and its value members
Student by reference / const reference	O(1) parameter setup	No Student object copy; const reference prevents modification

A returned-value version computes and returns a new fee in O(1), and the caller writes fee = applyConcession(fee, percent). For several concessions, compute from the current result or use a clearly specified combined percentage. Avoid assuming percentages can simply be added when they are applied sequentially.

Q2 – Return by Reference and Its Dangers

1. Safe reference to the highest mark

```
#include <iostream>
using namespace std;

int& highestMark(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}

int main() {
    int marks[] = {78, 91, 85, 88};
    highestMark(marks, 4) += 2;
    cout << marks[0] << " " << marks[1] << " "
```

```
        << marks[2] << " " << marks[3] << '\\n';
}
```

Output: 78 93 85 88. highestMark returns an alias to the array element containing the maximum. Therefore the returned expression is assignable and += 2 modifies the original array.

2. Dangerous local reference

```
int& invalidReference() {
    int local = 50;
    return local; // invalid: local is destroyed at function exit
}
```

A compiler normally warns that the function returns the address/reference of a local variable. After the function returns, local no longer exists. Using the returned reference is undefined behaviour; it may appear to work, print a random value, or crash. Do not use it.

3. Reference, const reference and copy

- int&: caller may modify the original object; the referred object must remain alive.
- const int&: caller cannot modify through this reference; avoids copying and is useful for read-only access.
- int: returns an independent copy; safe after the original object changes or is destroyed, but modifying the copy does not change the source.

4. Vector growth and invalidation

A std::vector may reallocate its storage when it grows beyond capacity. Reallocation moves elements to a new memory block and invalidates references, pointers and iterators to its old elements. A reference obtained before growth must not be used after a reallocation. Reacquire the element after growth, reserve sufficient capacity when appropriate, or use an index and access the vector again.

5. Alternative return types and complexity

```
int highestIndex(const int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return best;
}
// int i = highestIndex(marks, n); marks[i] += 2;

// Pointer alternative:
int* highestPointer(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return &marks[best];
}
// *highestPointer(marks, n) += 2;
```

Return form	Find maximum	Extra space	Safety note
int&	O(n)	O(1)	Convenient assignment; lifetime must be valid
int index	O(n)	O(1)	Often safest for vector; access again after growth
int*	O(n)	O(1)	Can be null/invalid; vector reallocation invalidates it

All three require a linear scan. An index is generally the most robust choice when the container may grow, because the vector can be indexed again after reallocation.

Q3 – Default Arguments and Function Overloading

1. Cause of ambiguity

The three proposed charge functions overlap. A call such as charge(2.0) can match the one-parameter overload, the two-parameter overload if its second parameter has a default, and the three-parameter overload if its remaining parameters have defaults. The compiler cannot select one unique best function in such a design. Remove overlapping overloads and provide one unambiguous kg function with defaults, plus overloads for genuinely different input types.

2. Redesigned implementation

```
#include <iostream>
using namespace std;

struct Parcel {
   double weightKg;
};

double charge(double weightKg, bool express = false,
              bool insured = false) {
   double amount = weightKg * 50.0;
   if (express) amount += 40.0;
   if (insured) amount += 20.0;
   return amount;
}

double charge(int weightGrams) {
   return charge(weightGrams / 1000.0, false, false);
}

double charge(const Parcel& p) {
   return charge(p.weightKg, false, false);
}

int main() {
   Parcel p{2.5};
   cout << charge(500) << '\\n';
   cout << charge(2.0) << '\\n';
   cout << charge(2.0, true) << '\\n';
   cout << charge(2.0, false, true) << '\\n';
   cout << charge(p) << '\\n';
   cout << charge(2.0f) << '\\n';
}
```

Assumption for this example: ₹50 per kg, ₹40 express surcharge and ₹20 insurance surcharge. These are illustrative rates because the assignment does not specify actual rates.

Call	Selected function	Reason / result
charge(500)	charge(int)	Exact match for grams; 0.5 kg → ₹25
charge(2.0)	charge(double,bool,bool)	Exact match for double; defaults false,false → ₹100
charge(2.0,true)	charge(double,bool,bool)	Exact first and second arguments; insured defaults false → ₹140
charge(2.0,false,true)	charge(double,bool,bool)	All arguments supplied → ₹120

charge(p)	charge(const Parcel&)	Exact Parcel overload → ₹125
charge(2.0f)	charge(double,bool,bool)	float promotes to double; defaults apply → ₹100

3. Overload resolution

- Exact match is preferred over a promotion or conversion.
- A promotion, such as float to double, is generally preferred over a standard conversion such as int to double.
- Default arguments fill omitted trailing parameters; they do not create a separate overload.
- Default arguments must be specified from right to left because arguments can only be omitted at the end of the call.

4. Alternative design using a single structure

```
struct ParcelOptions {
    double weightKg;
    bool express = false;
    bool insured = false;
};

double charge(const ParcelOptions& p) {
    return p.weightKg * 50.0
        + (p.express ? 40.0 : 0.0)
        + (p.insured ? 20.0 : 0.0);
}
```

Overload resolution occurs at compile time and has no run-time cost. The charge calculation is O(1) and uses O(1) additional space. A ParcelOptions structure is easier to extend with new options, while overloads provide convenient type-specific calls. Passing the structure by const reference avoids copying it.

Q4 – Specifying a Class with an Array Inside It

1. SavingsAccount class with a fixed transaction array

```
#include <iostream>
#include <string>
using namespace std;

class SavingsAccount {
private:
    long long accountNumber;
    double balance;
    double transactions[10];
    int count;

    void record(double amount) {
        if (count < 10) {
            transactions[count++] = amount;
        } else {
            for (int i = 1; i < 10; ++i)
                transactions[i - 1] = transactions[i];
            transactions[9] = amount;
        }
    }

public:
    SavingsAccount(long long number, double opening)
```

```
        : accountNumber(number), balance(opening), count(0) {}

    void deposit(double amount);
    bool withdraw(double amount);
    void miniStatement() const;
};

void SavingsAccount::deposit(double amount) {
    if (amount <= 0) return;
    balance += amount;
    record(amount);
}

bool SavingsAccount::withdraw(double amount) {
    if (amount <= 0 || amount > balance) return false;
    balance -= amount;
    record(-amount);
    return true;
}

void SavingsAccount::miniStatement() const {
    cout << "Account: " << accountNumber
         << "\\nBalance: " << balance << '\\n';
    cout << "Recent transactions:\\n";
    for (int i = 0; i < count; ++i)
        cout << transactions[i] << '\\n';
}

int main() {
    SavingsAccount a(1001, 1000);
    a.deposit(500);
    a.withdraw(200);
    a.deposit(100);
    a.miniStatement();
}
```

The transaction array, balance, account number and count are private so that callers cannot directly corrupt account state. Public methods enforce validation and maintain a consistent balance and transaction history.

2. Handling the 11th transaction

- Shifting design: when full, move entries 1–9 one position left and write the new transaction at index 9. Insertion costs O(k), here O(10), because k old entries are shifted.
- Circular-buffer design: maintain a start index and overwrite the oldest element, then advance start modulo capacity. Insertion is O(1). Printing in chronological order requires modular indexing.

For a last-50 requirement, changing the array size from 10 to 50 is simple, but shifting now moves up to 49 elements. A circular buffer is preferable when additions are frequent and the history size may increase.

3. Alternative using deque

```
#include <deque>
using namespace std;

deque<double> recent;
void addTransaction(double amount) {
    const size_t LIMIT = 10;
    if (recent.size() == LIMIT) recent.pop_front();
    recent.push_back(amount);
}
```

Design	Add when full	Storage
Fixed array + shift	O(k)	O(k), fixed capacity
Circular buffer	O(1)	O(k), fixed capacity
std::deque	O(1) amortised for push_back/pop_front at ends	O(k), dynamic container overhead

Q5 – Static Members and Arrays of Objects

1. Token prediction

The static variable next begins at 100. Three Token objects in counter are constructed first, receiving IDs 101, 102 and 103. The extra object is constructed fourth and receives 104. issued() returns next − 100 = 4.

```
101 103 104 4
```

The constructor runs four times. The static data member is shared by all Token objects, while id is an individual member of each object.

2. Complete the BusTicket code

```
BusTicket(const std::string& n) : seat(0), name(n) {
   if (issued < CAPACITY) {
      ++issued;
      seat = issued;
   }
}

static int count() {
   return issued;
}
```

Expected output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

The static member issued is defined outside the class exactly once: int BusTicket::issued = 0;. The static function count() can be called as BusTicket::count() without creating an object. It cannot use this or access an individual non-static member because it has no particular object.

3. Tickets created inside a function called twice

```
void createTickets() {
   BusTicket a("Esha");
   BusTicket b("Farhan");
   a.print();
   b.print();
}
int main() {
   createTickets();
   createTickets();
   cout << "Total issued: " << BusTicket::count() << '\\n';
}
```

The static issued value is not reset when createTickets returns. It persists for the lifetime of the program. With capacity 3, the first call issues seats 1 and 2; the second call issues seat 3 and puts the fourth request on the waiting list. The total remains 3.

4. Alternative counter designs

- Global variable: simple O(1) counter, but global state is exposed and can be modified by unrelated code.
- TicketOffice object: owns the counter and is passed to each ticket. This encapsulates the state and supports multiple buses by creating one office per bus.
- Static member: encapsulates one shared counter inside the class, but a single static counter is shared by all buses of that class. It is not suitable when independent bus capacities/counters are required.

All designs use O(1) time per ticket and one counter per shared scope. An office object is the most flexible choice for multiple independent buses.

Q6 – Friend Functions and Inline Functions

1. Completed friend functions

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time {
   int h, m;
public:
   Time(int hh, int mm) : h(hh), m(mm) {}
   friend int minutesBetween(const Time& a, const Time& b);
   friend bool earlier(const Time& a, const Time& b);
};

inline int minutesBetween(const Time& a, const Time& b) {
   int first = a.h * 60 + a.m;
   int second = b.h * 60 + b.m;
   return abs(first - second);
}

inline bool earlier(const Time& a, const Time& b) {
   if (a.h != b.h) return a.h < b.h;
   return a.m < b.m;
}

int main() {
   Time departure(9, 45), arrival(13, 10);
   cout << minutesBetween(departure, arrival) << ' '
        << boolalpha << earlier(arrival, departure) << '\\n';
}
```

Expected output: 205 false. 9:45 is 585 minutes after midnight and 13:10 is 790 minutes after midnight; the absolute difference is 205 minutes. arrival is not earlier than departure.

2. Correctness tests

```
Time same1(12, 0), same2(12, 0);
Time beforeNoon(11, 50), afterNoon(12, 10);
cout << minutesBetween(same1, same2) << '\\n';      // 0
cout << earlier(beforeNoon, afterNoon) << '\\n';    // 1
cout << earlier(afterNoon, beforeNoon) << '\\n';    // 0
```

3. Friend versus member function

- A friend is not a member function, but the class grants it access to private members. It is called as minutesBetween(a,b), treating both objects symmetrically.
- A member version would be called as a.minutesTo(b) and can directly access the current object's private data and the other object through the class interface.
- Public getters preserve private storage but expose read access. Making fields public weakens encapsulation and permits invalid values such as hour 27 or minute 90.

Friend functions are justified when a non-member operation naturally treats two objects equally and needs private access. Otherwise, member functions or public read-only getters usually provide tighter encapsulation.

4. Inline and performance investigation

The inline keyword requests that the compiler consider substituting the function body at the call site; it does not guarantee inlining. Modern optimising compilers decide based on function size, visibility and optimisation settings. Both functions perform a constant number of operations, so each call is O(1).

Build	What to measure	Measured time (fill after running)
g++ -std=c++17 -O0	10,000,000 calls, non-inline version	_____
g++ -std=c++17 -O0	10,000,000 calls, inline version	
g++ -std=c++17 -O2	10,000,000 calls, non-inline version	_____
g++ -std=c++17 -O2	10,000,000 calls, inline version	_____

Do not copy invented timing values into the report. Run both versions under the same compiler, machine, input and iteration count. At -O2, the compiler may inline both versions, so the difference can be very small or absent. At -O0, call overhead may be more visible. The time complexity remains O(1) per call and extra space is O(1).

Q7 – Diagnose, Redesign and Validate

Selected program: Q4 SavingsAccount transaction history. The original implementation shifts nine elements every time a new transaction arrives after the array becomes full. The work increases linearly with the history capacity k.

Q7 – Diagnose, Redesign and Validate

Selected program: Q4 SavingsAccount transaction history. The original implementation shifts nine elements every time a new transaction arrives after the array becomes full. The work increases linearly with the history capacity k.

A. Diagnose the Limitation

When the transaction history is full, adding one new item requires moving k−1 elements. For capacity 10 this is at most 9 moves; for capacity 50 it is at most 49 moves. The design performs unnecessary copying even though only the oldest entry needs to be discarded.

B. Define the Design Objective

Keep exactly the latest k transactions in chronological order, with O(1) time to record each new transaction and O(k) storage. The account balance and deposit/withdraw validation must remain unchanged.

C. Design the Revised Solution

```
class TransactionHistory {
private:
   static const int CAPACITY = 50;
   double data[CAPACITY];
   int start = 0;
   int count = 0;

public:
   void add(double amount) {
      if (count < CAPACITY) {
         data[(start + count) % CAPACITY] = amount;
         ++count;
      } else {
         data[start] = amount;
         start = (start + 1) % CAPACITY;
      }
   }

   void print() const {
      for (int i = 0; i < count; ++i)
         std::cout << data[(start + i) % CAPACITY] << '\\n';
   }
};
```

Pseudocode: if count is less than capacity, write at (start + count) modulo capacity and increase count. Otherwise overwrite data[start] and advance start by one modulo capacity. Printing begins at start and visits count entries using modular indexing.

The redesign uses the same O(k) memory as the fixed array and adds two integer fields. Insertion becomes O(1), but printing still takes O(k) and the modular indexing is slightly more complex.

D. Predict Before Implementing

For a full buffer of 50 items, the shifting version performs up to 49 element assignments for every new transaction. The circular buffer performs one data assignment and one start-index update. Therefore, insertion time should grow less with capacity and remain O(1) for the circular design.

E. Implement and Validate the Redesign

Test	Original shift array	Circular buffer	Expected check
Add 1–10	Stores 1 through 10	Stores 1 through 10	Both print 1…10
Add 11	Shifts 2–10, appends 11	Overwrites oldest and advances start	Both print 2…11
Add 51 to capacity 50	Shifts 49 entries	Overwrites oldest and advances start	Both print 2…51 after adding 51 values
Insertion complexity	O(k) when full	O(1)	Circular buffer avoids shifting

Record actual execution timings using the same compiler and test workload. Fill in the evidence table after running; do not present predicted values as measured results.

Capacity	Shift design measured time	Circular design measured time
10		
50		
500		

F. Defend the Final Design

The objective is achieved if tests confirm that both designs retain the same latest k transactions in the same order and the circular buffer performs a constant number of operations per insertion. The circular design is preferable for frequent updates or larger capacities. The shifting array remains easier to understand and may be adequate for a very small history with infrequent updates. The redesign trades simpler indexing for lower insertion work.

Conclusion

This assignment demonstrates parameter passing, reference safety, overload resolution, class encapsulation, static members, friend functions and inline functions. The selected redesign applies a circular buffer to reduce transaction insertion from O(k) to O(1) while preserving the latest transaction history.

Submission Checklist

- Create repository: DSA01-192425323-Assignment2.
- Create one folder for each task (Q1 to Q7) and save the C++ source files.
- Compile using a command such as: g++ -std=c++17 -Wall -Wextra program.cpp -o program
- Run every program and capture genuine output screenshots.
- For Q6 and Q7, fill the timing tables with measurements from your own computer.
- Add README.md with student details, compiler/version, compile/run commands and task summaries.
- Commit progressively, at least one commit per task, and add the working GitHub URL to the cover page.